ENSEIRB-MATMECA 1ère année Informatique

Projet Royal Dédale

Rapport de projet

Groupe I1 P5 - Équipe n°8
Ilan Vargas & Achraf Safi

19 décembre 2025



Table des matières

Introduction & Architecture **2**

1 Fondation **3**
1.1 Code source . . . 3
1.1.1 directions . . . 3
1.1.2 positions . . . 3
1.1.3 ressources . . . 4
1.1.4 poches . . . 4
1.2 Tests et validation . . . 5
1.2.1 Tests sur les directions et les ressources . . . 5
1.2.2 Tests sur les positions . . . 6
1.2.3 Tests sur les poches . . . 6
1.2.4 Conclusion sur la validité . . . 6
1.3 Conclusion . . . 6

2 Implémentation des tuiles **6**
2.1 Abstraction et interface du module `tile` . . . 6
2.2 Structure interne d'une tuile . . . 7
2.2.1 Définition statique des tuiles . . . 7
2.3 Représentation du plateau de jeu . . . 8
2.4 Opérations sur les tuiles . . . 8

3 Implémentation du jeu **10**
3.1 Manoir et joueur . . . 11
3.2 Boucle du jeu . . . 11
3.3 Exécution . . . 14
3.3.1 Le Makefile . . . 14
3.3.2 Paramètres d'exécution . . . 14
3.4 Visualisation graphique : TUI . . . 15

4 Limites et améliorations **15**

Conclusion **15**

Introduction

Le projet de programmation Royal Dédale consiste à implémenter un jeu d'exploration se déroulant dans un manoir représenté sous la forme d'une grille rectangulaire de taille fixe. À l'exception de la tuile de départ et de la tuile d'arrivée,

le manoir est initialement composé uniquement d'espaces vides, correspondant à des positions encore inconnues du joueur.

Le joueur débute la partie sur la tuile de départ et doit atteindre la tuile d'arrivée en explorant progressivement le manoir. Cette exploration repose sur l'ouverture de tuiles adjacentes aux positions déjà découvertes, puis sur le déplacement du joueur de case en case. Les tuiles sont ainsi placées dynamiquement au cours de la partie, ce qui modifie progressivement la configuration du manoir et confère au jeu sa structure labyrinthique.

Chaque tuile possède des caractéristiques propres, notamment des coûts de déplacement et des coûts d'ouverture selon les directions cardinales. Ces contraintes, associées à une gestion limitée des ressources et à un nombre maximal de tours, conditionnent les choix du joueur et influencent directement sa progression.

L'objectif de ce projet est de concevoir et d'implémenter une solution logicielle respectant strictement les interfaces abstraites imposées par le sujet, tout en garantissant le bon déroulement d'une partie dans le cadre de la version de base du jeu.

Afin de répondre aux contraintes du sujet, une architecture logicielle modulaire a été mise en place. La section suivante présente l'organisation générale du programme.

Architecture du programme

Nous détaillerons l'architecture du programme C de notre version de initiale de *Royal Dédale*

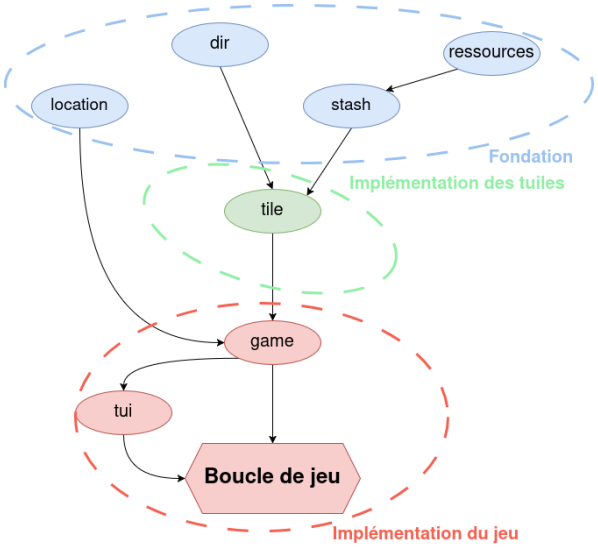


FIGURE 1 – Architecture de la version initiale de notre projet

La figure 1 nous montre les modules de nos programme et leurs interdépendances,

chaque flèche indique que le fichier au bout de la flèche est dépendant de celui à l'origine mais aussi de tout les modules dont ce dernier en dépend. Nous pouvons regrouper les modules en trois groupes :

- La **fondation**. Où on implémente les "types" (structures et énumérations) fondamentales pour notre programme : *localisation*, *direction*(*dir*), *ressource* et *stash*(*poche*) ; et leurs opération associées. Ces conceptes servent de base à la quel s'appuie les modules suivents.
- L'**implémentation des tuiles**. Avec les modules précédents nous créeons les tuiles et leurs opérations. C'est de ça qui est composé le manoire implémenter par la suite et une mécanique fondamentale de la partie.
- L'**implémentation du jeu**. C'est la que nous créons le manoir, les fonctions pour le manipuler, des fonction pour l'afficher en forme d'interface textuelle et combinons tout ça dans une boucle qui nous permettra de jouer le jeu. Dans cette étape nous combinons tout notre code précédant pour créé finalment le jeu.

Cet architecture donne un jeu fonctionnel mais forcément de la manière la plus optimal ou complet en terme de jouabilité, nous verrons les limites et les amélioratoins proposé à ce programme. Mais avant ça, nous commençons par detailler la fondation de cette architecture.

1 Fondation

Nous detaillons les "types", et leurs opérations, utilisé comme base de ce projet.

1.1 Code source

1.1.1 directions

On défini un type enuméré dir_t afin de gérer les différentes directions cardinales (nord, sud, west, oest) d'une manière simple et claire tout au long du project.

Le module implémente les fonction `dir_to_string` qui représente une direction par la chaîne de caractère qui lui est associée, ainsi que la fonction `dir_to_short_string` qui renvoie la représentation d'une direction par une petite chaîne de caractère.

De plus, il implémente la fonction `dir_opposite` qui permet d'obtenir la direction opposée à celle fournie en entrée.

1.1.2 positions

On définie la position par une structure $location_t$ contenant un couple de coordonnées (x, y) correspondant respectivement à l'abscisse et l'ordonnée d'une

position dans le manoir. Une fonction `location` est fournie ainsi afin de générer cette structure à partir du couple de coordonnées fournies.

Le module inclut les fonctions nécessaires pour manipuler les positions dans le manoir :

— `location_is_valid`, verifie si une position est bien dans le manoir (dans les limites).

— `location_equal`, verifie si deux position a et b sont égales.

— `location_is_adjacent`, pour une position a verifie si la position b est adjacente.

Les deux dernière renvoient faux si une des position n'est pas valide.

1.1.3 ressources

Cet interface permet de définir le type énuméré $resource_t$ qui représente l'ensemble des types de ressources dans le jeu dont le nombre est fixé à $max_resources = 10$, cette implémentation simplifie parsuite le stockage et la manipulation des ressources du joueur dans une poche, décrite dans la section suivante.

Entre les ressources differentes disponible, la plus notables sont les $steps(pas)$. Jusqu'à la version amélioré, c'est la seul ressource utilisé : pour chaque mouvement du joueur ou plaçage de tuile on consomme une *step* et donc on a besoin d'avoir au minimum un *step* pour faire une action pendant sont tour sinon on est coincé.

L'implémentation inclut des fonctions qui permettent de représenter la ressource en chaine de caractère, soit `resource_to_string` pour le nom complet de la ressource et `resource_to_short_string` pour une représentation minimale (généralement un caractère).

L'implémentation de ces fonctions est réalisée grâce à la structure $switch-case$ afin de gérer le cas de toutes les ressources.

1.1.4 poches

Après avoir défini les types de ressources via une énumération, nous implémentons maintenant le module *stash* (poche) qui constitue le système gestion des ressources du joueur.

On introduit la structure $stash_t$, qui représente une poche de ressources. Il s'agit d'un tableau dont la taille correspond exactement au nombre de types de ressources en jeu où chaque case du tableau stocke la quantité de la ressource associer à son indice, selon $resource_t$. Après avoir implémenter les ressources dans le module précédant, on implémente un module

Ce fichier regroupe l'ensemble des opérations sur les poches :

— `stash_add`, l'addition de deux poches a et b : retourner une poche contenant pour chaque ressource l'addition du nombre de cette ressource dans a et b

— `stash_sub`,la soustraction d'une poche a à une autre b, pour chaque ressource de a on la soustrait à celle de b (0 si la valeur est négative)
— `stash_contains`, verifie si une poche b est contenue dans une poche a (pour toutes les ressources de b, ça quantiter est plus petite ou égale à celle de a ; deux poches égales ce sontiennent)

On a une fonction supplémentaire pour une représentation en chaine de caractère de la poche `stash_to_string`, utile pour une interface du jeu.

1.2 Tests et validation

Dans cette partie on s'intéressera à l'implémentation des tests et comment ils permettent de valider la correction des fonctions.

Généralement, les fonctions du code source sont testées sur plusieurs entrées ce qui demande la création de trois tableaux, un tableau pour les entrées, un pour les sorties et un autre contenant les solutions attendues, pour comparer la sortie à la solution. Un exemple de ces tests sera fourni dans ce qui suit.

1.2.1 Tests sur les directions et les ressources

Les tests sur ces deux modules servent à vérifier la sortie des deux fonctions de conversion des directions et ressources en leurs noms sous forme de chaine de caractère, donc la validation des tests correspond à la verification de l'égalité entre la chaine de sortie et celle attendu.

D'autre part, le test `dir_oposite` demeure simple grâce au type énuméré des directions, il suffit de vérifier l'égalité entre la sortie et la solution autant que deux entiers.

```
TESTS ON resource.c
Tests on resource_to_string
PASSED  input:   0   output: NOTHING
PASSED  input:   1   output: STEP
PASSED  input:   5   output: SILK
PASSED  input:   9   output: OIL
PASSED  input:  10   output: CRYSTAL
Tests on resource_to_short_string
PASSED  input:   0   output: N
PASSED  input:   1   output: St
PASSED  input:   4   output: Gd
PASSED  input:  10   output: C
```

Code 1 – Exemple des tests sur les ressources

1.2.2 Tests sur les positions

Afin de valider les tests sur les fonctions manipulant les positions du manoir, appelons les fonction sur des positions differentes dans le but de tester tout les cas pertinents, notamment les postions dedans et dehors le manoire pour tester la validité de la position du manoir ; des couples égaux adjacents ou pas pour les fonctions qui comparent deux positions

1.2.3 Tests sur les poches

Avant de procéder aux tests, une fonction est implémentée afin de générer des poches dont les ressources sont des valeurs aléatoires inférieures à 10. Cette fonction permettera ensuite d'observer facilement les quantités de chaque ressource de la poche retournée en les comparant à celles des poches passées en argument, et ainsi de déduire si les opérations effectuées correspondent bien aux opérations élémentaires appliquées ressource par ressource.

1.2.4 Conclusion sur la validité

Même si nous ne pouvons pas tester toutes les combinaisons posibles pour chaque fonction, on deduit qu'on a tester assez de cas pertinents differents pour assumer les fonctions valide et poser notre programme dessu.

1.3 Conclusion

L'ensemble des fichiers du code source présentés dans cette section ne présente pas de difficultés particulières en termes d'implémentation ou de complexité mais ils sont toutefois essentiels, car ils permettent de définir et de structurer les données utilisées par la suite. Le découpage du code en plusieurs fichiers, associé à la compilation séparée, facilite le travail en équipe. Chaque module peut être développé, modifié ou amélioré indépendamment des autres, tout en respectant les interfaces définies par les fichiers d'en-tête. Cette organisation permet ainsi d'intégrer différentes implémentations d'un même module sans impacter le reste du programme.

Nous verrons maintenant comment passer à l'étape suivante l'implémentation des tuiles.

2 Implémentation des tuiles

2.1 Abstraction et interface du module `tile`

La gestion des tuiles est assurée par le module `tile`, dont l'interface est définie dans le fichier `tile.h`. Ce module repose sur une structure abstraite `tile_t`, dont la

définition interne est volontairement masquée au reste du programme. Cette abstraction permet de découpler l'utilisation des tuiles de leur implémentation concrète, et garantit que les autres modules du jeu ne dépendent que de l'interface publique.

Grâce à ce choix, il est possible de modifier l'implémentation des tuiles dans `tile.c` (ajout de nouvelles tuiles, modification des coûts d'ouverture et de traversage) sans nécessiter de changements dans le reste du code, tant que les fonctions exposées par `tile.h` conservent le même comportement.

2.2 Structure interne d'une tuile

Dans le fichier `tile.c`, la structure `tile_t` est définie par plusieurs attributs caractérisant une tuile du jeu :

— un *nom*, pour identifier la tuile
— un *cotdetraverse* sous forme de poche, payé à chaque passage du joueur sur la tuile
— un tableau de *cotsd'ouverture* sous forme de poches, indexé par direction, indiquant si la tuile peut être ouverte pour chaque direction et le coût
— un *poids*, utilisé lors de la génération aléatoire des tuiles.

Cette représentation permet de décrire de manière uniforme l'ensemble des tuiles du jeu.

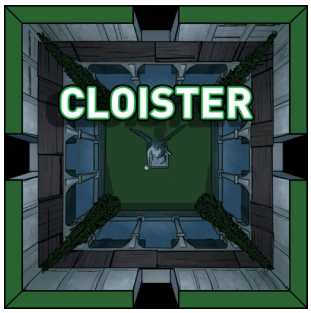
2.2.1 Définition statique des tuiles

Les différentes tuiles du jeu sont définies comme des instances statiques de `tile_t` dans le fichier `tile.c`. Cette approche évite toute allocation dynamique inutile et garantit que les caractéristiques des tuiles sont centralisées en un seul endroit.

On définit, dans notre versoin, les tuiles spéciales : tuile vide `EMPTY_TILE`, tuile de départ `START_TILE` et tuile d'arrivée `EXIT_TILE`. Puis les tuiles a placé pendant la partie :

— `CLOISTER_TILE`, s'ouvre dans toutes les directions, a un poid égale à 2 (plus de chance d'être généré)
— `EW_CORRIDOR_TILE & NS_CORRIDOR_TILE`, des couloir c'est-à-dire ouverte selon deux direction opposé.Il y en a deux car une pour ouverte selon la direction nord-sud et l'autre est-ouest faisant ainsi toutes les rotations de couloires
— les tuiles térraces, ouverte à peine selon une direction. On impémente une une tuile par direction

Pour l'instant tout coût d'ouverture ou traversage est égale à 1 *step*(*pas*) pour cela il faut avoir au moins 1 *step* pour pouvoir jouer. Si une tuile n'est pas ouverte selon une direction, le coût d'ouverture de tel direction pour la tuile est une poche contenant UINT_MAX c'est-à-dire l'entier maximal en C.




```c
struct tile_t CLOISTER_TILE = {
  .name = "cloister",
    .traversal_cost = ONE_STEP_STASH, //poche contenant un pas
  .opening_cost = {
    ONE_STEP_STASH,
    ONE_STEP_STASH,
    ONE_STEP_STASH,
    ONE_STEP_STASH
  },
  .weight = 2
};
```

Code 2 – Exemple d'implémentation d'un cloister

2.3 Représentation du plateau de jeu

Le plateau de jeu est représenté par une matrice de pointeurs vers des structures `tile_t`. Chaque case du plateau référence donc une tuile existante, sans en contenir une copie. Ce choix présente plusieurs avantages :

— il évite la duplication de structures potentiellement volumineuses
— il réduit la consommation mémoire
— il simplifie les opérations de lecture et de modification des tuiles
— il permet de comparer des tuiles via leurs adresses

On vera ça plus en détaille dans la section suivante.

2.4 Opérations sur les tuiles

Les différentes tuiles du jeu sont regroupées dans un tableau de pointeurs *p_tiles* vers les implémentations statique. Cette organisation facilite l'implémentation des opérations.

Les structures étant implémentées de manière statique et manipulées uniquement via des pointeurs, elles ne sont pas accesibles en dehors `tile.c`. Les interactions avec les

tuiles (lecture des coûts, vérification des ouvertures, génération, etc.) sont donc réalisées à l'aide de fonctions dédiées, définies dans l'interface du module.

Nous presentons les opérations faites sur les tuiles avec pour entrée un pointeur pointant sur une tuile statique de `tile.c`.

tile_name

Cette fonction retourne une chaîne de caractères correspondant au nom de la tuile pointée par le pointeur passé en paramètre.

tile_traversal_cost & tile_opening_cost

Retourne une poche correspondant au coût de traversé, ou d'ouverture selon une direction respectivement, d'une tuile pointée par le pointeur passé en paramètre.

tile_generate_empty, tile_generate_start & tile_generate_exit

Ces fonctions retournent le pointeur vers la tuile vide, de la tuile de départ et de la tuile de sortie, respectivement. On utilisera ces pointeurs pour construire le manoir.

tile_generate_by_name

Retourne le pointeur vers la tuile de nom passé en paramètre sous forme de chaine de caractère.

Pour cela, pour chaque pointeur de p_tiles on verifie si le nom de la tuile associée est égal à celui passé en paramètre. Si oui, on retourne le pointeur.

tile_is_empty

Verifie si le pointeur d'entrée pointe vers la tuile vide, c'est-à-dire si la tuile en entrée est la tuile vide.

tile_can_open

Verifie si la tuile tuile passé en paramètre autant que pointeur peut être ouverte selon une direction passsé en paramètre également.

Pour ca on verifie si le coût d'ouverture de la tuile selon la direction est une poche avec pour nombre de steps inférieur à U_INT_MAX

tile_draft

Pour une direction en paramètre on retourne au maximum NUM_TILES_PER_TURN pointeur de tuile, avec un buffer, choisis de manière aléatoire pouvant être ouvert dans la direction en paramètre.

Avant de faire ça on appel au début du jeu une fonction `init_tile` qui initialise la graine aléatoire pour pouvoir utiliser l'aléatoire pour selectionnner les tuile choisis et puis initialise une matrice statique *p_tiles_direction*. Cette matrice contient quatre tableaux, une par direction cardinale, et chaque tableau contient toutes les pointeur de tuiles pouvant être ouvert selon la direction associée à l'indice du tableau dans la matrice.

`p_tiles_direction[NORD]` est le tableau de pointeur de toutes les tuiles qui sont ouverte au nord.

Dans la fonction `tile_draft` on répète NUM_TILES_PER_TURN fois :

On choisi un pointeur de tuile aléatoirement dans le tableau *p_tiles_direction* correspondant à la direction passé en paramètre. Si elle a déjà été choisi, c'est-à-dire déjà présente dans le buffer, alors on ne la remet pas, sinon ou la place dans le buffer.

Celle-ci est la fonction la plus compléxe du module.

En temps, on verifie à chaque fois si la tuile n'a pas été répéter, ce qui engendre une compléxité de

$$\Theta(1 + 2 + ... + N) = \Theta(\frac{N(N-1)}{2}) = \Theta(N^2) \tag{1}$$

Avec N = NUM_TILES_PER_TURN. Le processus pour verifier si une tuile est répété est une boucle dans une boucle.

On améliore la compléxité en temps ne choisisant pas une tuile aléatoirement entre toutes les tuile jusqu'à qu'elle s'ouvre dans la bonne direction. Plustôt, o choisit parmis les tuiles ouvrable selon tel direction, ce qui entraine, néanmoins, un coût spacial liée aux quatre tableaux statiques *p_tiles_direction*. On a pas besoin de beaucoup s'en inquiéter ici car NUM_TILES_PER_TURN=3 et on a 7 types de tuiles ce qui ne sont des chiffres relativement petits et donc une compléxité manageable.

On remarque que la quantité de tuiles choisi par la fonction est variable, pour cela on retourner le nombre de tuile choisies.

3 Implémentation du jeu

Á présent on a le code pour crée le jeu.

3.1 Manoir et joueur

On définit le manoir et le joueur par une structure qui permet de mémoriser l'état du jeu en tout moment.

```
struct game_state_t {
  struct stash_t steps;
  struct tile_t* board[BOARD_WIDTH][BOARD_HEIGHT];
  struct location_t current_location;
};
```

Code 3 – Structure du jeu

On garde dans cette structure les ressources du joueur dans la poche `steps` et sa position dans le manoir `current_location`.

Le manoir lui est une rectagle de taille (BOARD_WIDTH, BOARD_HEIGHT) composé de d'adresses de tuiles comme abordé dans la sectoin précédante. On stocke le type de tuile de chaque position dans la matrice `board`.

Avant de commencer la boucle de jeu, nous créerons une instance de cette structure appellé `game_state`. Elle permettra de définir l'état de notre jeu et sera le seul appel de cette structure.

3.2 Boucle du jeu

Après avoir définit l'état de notre jeu `game_state` on peut passer à la boucle de jeu, c'est-à-dire le déroulement de la partie.

```
init_tiles(t);
init_tiles_bis(t);
init_state(s, &game_state);
init_tui(is_graphic);

while (current_turn <= n && !is_player_won && game_state.steps.r[
   STEP] > 0)
{
    printf("\e[1mTurn number %d out of %d\e[0m\n", current_turn,
       n);
    int num_possible_locations =
       list_empty_locations_with_finite_opening_cost(
       possible_locations, &game_state);
    if (current_turn == 1 || (num_possible_locations &&
       decide_to_place_tile(is_iterative)))
    {
        printf("Place your tile\n");
        struct location_t selected_location = select_location(
           is_interactive, possible_locations,
           num_possible_locations);
        int num_available_tiles = generate_some_tiles(
           selected_location, available_tiles, &game_state);
        struct tile_t* selected_tile = select_tile(is_interactive
           , available_tiles, num_available_tiles);
        place_tile(selected_location, selected_tile, &game_state)
           ;
        tui_place_tile(is_graphic, selected_location,
           selected_tile);
    }
    else
    {
        printf("Move the player\n");
        int num_possible_locations = list_open_location(
           possible_locations, &game_state);
        struct location_t selected_location = select_location(
           is_interactive, possible_locations,
           num_possible_locations);
        tui_move_player(is_graphic, selected_location, &
           game_state);
        move_player(selected_location, &game_state);
    }
    current_turn++;
    tui_print(is_graphic);
    is_player_won = exit_reached(&game_state);
}
```

Code 4 – Implémentation de la boucle du jeu

Analyson le code du code 4

Avant de rentrée dans la boucle on initialise tout nos modules. En particulier on initialise `game_state` c'est-à-dire : on définit la position du joueur en $(0, 0)$; on pose toutes les tuiles du manoire comme vide sauf les tuiles en $(0, 0)$ la tuile de départ et (BOARD_WIDTH-1, BOARD_HEIGHT-1) la tuile d'arrivé ; on définit les ressources du joueur comme ayant $4 * 4 * \frac{BOARD_SIZE}{3}$.

Dans la boucle, à chaque itération, on vérifie que le nombre maximal de tours n'a pas été dépassé, que le joueur n'a pas encore gagné la partie et qu'il dispose de ressources suffisantes pour continuer à jouer (c'est-à-dire au moins un *step* pour faire une action). Sinon la partie est terminer, le joueur à gagné s'il a atteint la tuile d'arrivé sinon perdu.

Pour chaque tour, `list_empty_locations_with_finite_opening_cost` cherche et stocke dans un buffer les localisation vide ouvrables depuis la position du joeur (adjacente et ouverte dans la direction opposé du joueur). Si il a trouver au moins une on donne au joueur la possibilité de placer une tuile dans une de ces position ou bouger, sinon on l'oblige a bouger. Le joueur est obligé de placer le premier tour.

Si le joueur place

1. Le joueur choisit une des tuile vides disponibles
2. `tile_draft` génère des tuiles compatibles à être placé à la place de la tuile vide dans `generate_some_tiles`
3. Le joueur choisit une de ces tuile
4. On remplace la tuile vide par la tuile choisit dans `game_state`. On fait le joueur payer les coûts associés.

Dans le jeu, on manipule des positions *location* alors que les fonctions du module tile travaillent avec des directions. Pour faire le lien, nous avons créé la fonction `location_to_dir` dans `generate_some_tiles`. Cette fonction prend la position actuelle du joueur et une position adjacente, et retourne la direction correspondante en comparant la position donnée avec les positions situées au nord, au sud, à l'est et à l'ouest du joueur à l'aide d'une structure switch ou d'une boucle.

Si le joueur bouge

1. On liste toutes les listes dont le joueur peut bouger, c'est-à-dire la tile du joueur et la tuile listée sont ouvertes uen vers l'autre et adjacentes
2. Le joueur séléctionne une de ces tuiles
3. On bouge le joueur, update `game_state` et faisons payer le joueur les coûts associées.

Chaque "choix" peut se faire de manière aléatoire uo le joueur (derrière l'écran) peut faire les choix avec le mode itérative (expliqué plus bas).

On remarque que de nombreuses fonctions de la boucle principale nécessitent l'accès à l'état courant du jeu pour fonctionner correctement. Pour cette raison, l'adresse de la structure `game_state` est passée en paramètre à ces fonctions, ce qui leur permet d'accéder directement à l'ensemble des informations nécessaires.

Cette approche améliore la lisibilité et la maintenabilité du code : au lieu de transmettre séparément le manoir, la position du joueur ou ses ressources, l'intégralité de l'état du jeu est centralisée dans une seule structure.

De plus, le passage par adresse permet aux fonctions de modifier directement l'état du jeu et d'accéder aux données sans surcoût inutile. En particulier, la matrice board n'est pas recopiée à chaque appel de fonction ; elle est simplement lue ou modifiée via `game_state`, ce qui évite des opérations coûteuses et des complexités supplémentaires.

3.3 Exécution

3.3.1 Le Makefile

Un *Makefile* est un fichier utilisé par la commande *make* pour automatiser la compilation du projet selon des règles.

Notre make file à pour règles :

— `%.o: %.c`, règle générique qui compile chaque fichier source en fichier objet.
— `tester:` *fichier objet*, crée un exécutable capable d'exécuter les tests du projet
— `project:` *fichier objet*, crée le fichier de jeu exécutable à partir des fichier source compilé en objet grâce à la première règle
— `test:` , exécute le exécutable test *tester*
— `compile: project tester`, crée l'exécutable de jeu et de test
— `all:` , règle compile

3.3.2 Paramètres d'exécution

L'analyse de la ligne de commande pendant l'exécution se fait au moyen de la bibliothèque *getopt*, cela permet pour un même exécutable de changer les paramètres essentiels suivants :

— rendre le jeu interactif afin de pouvoir joueur : **-i**.
— graine aléatoire pour prendre les décision à la place du joueur (mode non interatif) : **-s**.
— graine aléatoire `tile_draft` : **-t**.
— fixer le nombre maximum de tours de jeu possible : **-n**.

Une autre fonctionalité à été mise en place pour une meilleur expériance de jeu :

— visualisation graphique tui du manoir : **-g**

3.4 Visualisation graphique : TUI

Une des fonctionnalités intéressantes à ajouter au projet était la TUI (Text User Interface), cet outil permet la visualisation du jeu directement dans le terminal. Le manoir est affiché sous forme d'une grille textuelle constitué des cases qui sont représentées sous la forme d'un tableau de caractères, l'implémentation consiste alors d'initialiser le manoir rempli par les cases vides et de placer chaque tuile dans sa case tout en respectant ses directions d'ouverture avec une mise à jour au fur et à mesure de l'avancement du jeu. La TUI offre également une représentation de la position du joueur sur le plateau, mise à jour à chaque déplacement.

Cette interface ne gère pas la logique du jeu, mais fournit une vue cohérente de son état courant, facilitant ainsi l'observation du déroulement de la partie et la validation du comportement du jeu lors des tests d'exécution. la figure ci-dessous illustre un exemple de la visualisation du manoir.

4 Limites et améliorations

Conclusion